

The Expression Is Not Enough: Executed-Ground-Truth Probes of Python Binding Semantics in gemma-2-2b

Anna Liang

PyCon TW 2026 poster project, companion note

2026-08-27

Abstract

A companion mechanistic-interpretability project established that `gemma-2-2b` carries a clean, causally verified representation of *what a Python local name is bound to*: one pathway per name, orthogonal across names, with dose-dependent control of which variable a function returns. This note asks the follow-up question: does the model also represent *when* a binding happens, *in which scope*, and behaviours where the printed expression underdetermines the answer? We add two behavioural probes (31 scored single-token cases) with three methodological commitments: ground truth is obtained by *executing* every snippet under the exact interpreter and environment the case names; every case sits in a pair of near-identical surface form and opposite answer, so form-matching cannot score both; and the harness reports candidate coverage, exact ties, and primer-order dependence rather than letting argmax and pooled accuracy flatter the model. The model fails almost everything that requires more than the surface: it is *worse* on a generator expression than on its PEP 289-defined desugaring (one program, written twice); it answers float comparisons by literal identity rather than arithmetic (`1e500 == 1e600` $\rightarrow$ `False` at 67.6%); it assumes sets preserve insertion order (97.3% on a hash-table wrap); and it gives the same `locals()` answer for Python 3.11, 3.12 and 3.13, versions across which the answer changed. Overall: 1 of 13 pairs pass. We argue the harness findings — a non-answering failure mode caught by coverage, a primer-dependent “classic” (`0.1 + 0.2 == 0.3`), and argmax silently awarding exact ties — are as reusable as the model findings.

1 Introduction

The parent project attributed a next-token decision — *which local variable does a Python function return?* — to per-name feature pathways in `gemma-2-2b`, and verified the attribution causally: scaling one pathway down flips the returned name, monotonically, crossing at ≈ 0.35 of the feature’s natural activation. That result concerns *binding as a static fact*: which name

was assigned.

An AI code debugger needs strictly more: *when* a binding takes effect (eager vs. lazy evaluation), *which scope* resolves a name, and cases where the expression on the page underdetermines the value (float width, hash-table order, allocator behaviour, interpreter version). This note probes those behaviourally. The contribution is deliberately two-sided: measured results for one small model, and a probe harness whose honesty mechanisms were each forced by a real failure we observed.

2 Method

Single-token divergence cuts. Every case is a prompt cut at a position where the correct semantics and a natural misreading predict *different next tokens*, all single tokens under the Gemma tokenizer. One forward pass decides each case; we read the full softmax and report candidate probabilities, not samples.

Executed ground truth. No expected output is asserted from memory. Each case carries runnable source; the harness executes it in a subprocess — under the exact interpreter (`python3.11/3.12/3.13` where the case names one) and environment (e.g. `PYTHONHASHSEED=0`) — and refuses to start unless (i) the prompt is a true prefix of what Python prints, (ii) the candidate labelled correct is the actual next character, and (iii) the distractor is not. Chapters probing CPython behaviour that the language does not guarantee (§4, ch. 3) are marked implementation-defined: a mismatch warns and skips rather than aborting, and their results are never quoted as statements about “Python”.

Paired scoring. Cases come in pairs of near-identical surface form with opposite correct answers. A model that matches on form scores at most one member; only passing both counts. Pooled per-case accuracy is reported only to show how much it flatters the model.

Verdict hygiene. A case is *noise* if both candidates fall below a 2% floor (the model’s answer is neither candidate, so comparing them compares scraps); a *tie* if the two probabilities are exactly equal — argmax would otherwise award the case to whichever candidate is listed first, and the correct one always is; and *prime-dep* if its verdict flips with the order of the primer exchanges (below). The summary prints *coverage* — $P(\text{correct}) + P(\text{wrong})$ — for every case.

Counterbalanced priming. On bare `>>> expr` prompts the model’s top prediction was `>>> itself` at 37–45%: it declined to answer and moved to the next prompt, leaving both candidates in the noise. Affected prompts are therefore prefixed with two worked exchanges (one `False`, one `True`), run under *both* orders. Coverage rose to 92–98%. A single primer order would have been an uncontrolled bias; the counterbalance caught three cases whose verdict it would have silently set.

Setup. `google/gemma-2-2b`, plain transformers forward passes, bfloat16 on an RTX 3090 (float16 overflows gemma-2’s activations). No transcoders or attribution are needed for anything in this note. One model only; nothing here is claimed to generalise across models.

3 Experiment 1: generator-expression late binding (PEP 289)

PEP 289 defines a generator expression’s outermost iterable to be evaluated *eagerly*, at definition time, and every other name *lazily*, at iteration time. The probe material is real (snippets from a teaching video, kept precisely because synthetic prompts invite “you designed that”):

```
lst = [1, 2, 3, 4, 5]
g = (n for n in lst if n in lst)
lst = [0, 1, 2]
print(list(g))          # -> [1, 2]
```

The iterable was captured eagerly (old `[1..5]`); the filter resolves the global lazily (new `[0,1,2]`); the output is their intersection — neither list on the page. Crucially, PEP 289 *defines* this expression to mean its desugaring (an explicit generator function taking `iter(lst)` as an argument and filtering on the global), so the two forms are one program written twice. The probe cuts both forms at the same two points and checks whether the model answers them alike: a disagreement rules out reading of the semantics outright, while agreement proves little, since two matching wrong answers also agree.

Results (Table 1). The model is *worse on the form a human would actually write*, on both cuts:

cut (correct)	genexp	desugared
iterated list ('1')	'0' 55.1% vs '1' 37.7% — miss	40.4 / 40.4 exact tie
filter’s list ('J')	53.1% — ok, margin 6.3 pp	73.1% — ok, margin 46.1 pp

Table 1: One program, two spellings. The model is worse on the sugared form on both cuts.

confidently wrong about which list is iterated where the desugaring cannot decide at all, and right by 6.3 pp about the filter where the desugaring is right by 46.1 pp. Controls rule out the cheap explanations: at length five with no reassignment it is correct at 99.6%, so closing the list early is not a taste for short lists; merely adding the vacuous filter `if n in lst` — before any rebinding — costs 10 pp (89.5%). On the parent project’s original cut that cleanly separates eager from lazy capture (whether `(3, ...)` appears after the inner list is exhausted), the correct continuation receives **0.00%**.

Stated carefully: this *supports* surface-form reading over semantics but does not establish it. The design catches an outright contradiction between the two forms, and it did not get one — an exact tie has no answer to contradict.

4 Experiment 2: four chapters of “the expression is not enough”

Twenty-six scored cases in thirteen pairs, in four separately scored chapters — separately, because pooled accuracy across a float-arithmetic chapter and an interpreter-versioning chapter measures nothing. **1/13 pairs pass** (list-vs-set order, the least surprising).

Ch. 1: float width (0/6 pairs). `1e500` is not a big number; the literal overflows to `inf` at parse time, and `1e500/1e500` is `nan`. Twelve cases, six pairs, all twelve verdicts consistent with a single heuristic: *literal identity, not arithmetic*. The same expression written twice gets `True` — correctly for `1e500 == 1e500` (78.9%), incorrectly for `1e500/1e500 == 1e500/1e500` (83.2%; it is `nan`, hence `False`). Two different literals get `False` — correctly for `1e50 == 1e60` (69.1%), incorrectly for `1e500 == 1e600` (67.6%; both are `inf`, hence `True`). Nothing was computed.

Ch. 2: hash order vs insertion order (1/3 pairs). The sharpest minimal pairs in the set: a dict comprehension and a set built from the same elements, both printed with braces, opposite first element. On `set(range(1022, 1027))`, whose repr starts at 1024 because the hash table wraps, the model puts 97.3%

on 1022-first: it assumes sets preserve insertion order the way dicts do. A deterministic-but-underivable case (string set under PYTHONHASHSEED=0) is included only as a floor and expected to miss; the unseeded variant is executed under two seeds to prove its output differs, and is therefore *never shown to the model* — there is no correct token to score.

Ch.3: bound-method identity (0/3 pairs; CPython, not Python). `o.f` constructs a fresh bound method per attribute access, so `o.f is o.f` is `False` while `C.f is C.f` and `o.f == o.f` are `True` — and `id(o.f) == id(o.f)` is `True` only because the allocator hands the freed address straight back, which one interleaved allocation flips to `False` (both stable 5/5 on this build). The model answers `True` to `o.f is o.f` (53.5%) and `True` to both `id()` variants — getting the reuse case right for the wrong reason and its control wrong, while `o.f == o.f` lands on an exact 44.1/44.1 tie.

Ch.4: interpreter version (0/1 pair). The same snippet — `locals()` inside a comprehension, with a cell variable — prints `{'.0': ..., 'k': 0}` on Python 3.11 and `{'k': 0}` on 3.12/3.13 (PEP 709 inlined comprehensions). Ground truth was executed on all three interpreters. The prompt is a shell transcript naming the interpreter; only that token differs. The model answers `'k'` for *every* version (21–23%), with `'.'` at 0.00% — right for 3.12/3.13, and structurally incapable of the 3.11 answer. It has one answer per snippet, not a model of interpreter versions.

5 Methodological findings

Three harness results we consider as reusable as the model results:

1. **Coverage, or the model may not be answering.** On bare `>>>` prompts the top token was `>>>` (37–45%). Any harness that reads only its two candidate probabilities would have scored this state as meaningful. Printing $P(\text{correct}) + P(\text{wrong})$ per case makes the failure mode impossible to hide; one low-coverage verdict in ch.2 (19.4%) is explicitly flagged as resting on scraps.
2. **Counterbalanced priming, or the context answers for the model.** With a False-first primer, `0.1 + 0.2 == 0.3` is an exact 48.83/48.83 split; with a True-first primer it answers `True` at 74.2%. The verdict belongs to the primer, not the model, and is reported as *prime-dep* (three cases; a single fixed primer would have silently decided all three).
3. **Exact ties, or argmax invents a preference.** `bfloat16` makes exact ties common (40.43/40.43,

44.14/44.14, 48.83/48.83 in this work alone). Argmax resolves them by candidate order — and the correct candidate is always listed first, so every such coin flip would have been recorded as a hit.

6 Limitations

One 2B model; behavioural probes only (no attribution was run for these cases); single-token cuts probe recognition, not generation; the genexp result supports surface-form reading but, absent a contradiction between the two spellings, does not establish it; ch.3 is CPython allocator behaviour and is not evidence about the language. No cross-model claims are made.

7 Reproducibility

Everything is in the project repository: probes `experiments/probe_late_binding.py` and `experiments/probe_gotchas.py`; full per-token transcripts under `experiments/outputs/`; real-code material with provenance under `experiments/python_code_material/`. Both probes verify their own ground truth by execution before loading the model and refuse to run otherwise. The parent project’s setup (environment, pinned dependencies, attribution pipeline) is documented in `experiments/README.md` and `HANDOFF.md`; the poster lives in `poster/`.